

Proyecto: Generador de Preguntas Inteligente con Ollama

Descripción general

Desarrollarás una **aplicación full-stack** que genera preguntas educativas automáticamente usando un modelo de lenguaje local (Ollama). El backend en Express se conecta a Ollama para generar preguntas sobre temas predefinidos, las almacena en una base de datos SQLite3, y un frontend en JavaScript vanilla permite seleccionar temas y visualizar las preguntas generadas.

El objetivo es comprender cómo integrar un servicio de IA local con una API REST, gestionar persistencia de datos y construir una interfaz funcional.

Objetivos de aprendizaje

- ✓ Configurar un proyecto Node.js con Express de forma modular
 - ✓ Conectar e integrar Ollama como servicio de generación de texto
 - ✓ Diseñar prompts efectivos para generar contenido educativo de calidad
 - ✓ Gestionar bases de datos con SQLite3 (mejor-sqlite3)
 - ✓ Construir una API REST con endpoints claros y documentados
 - ✓ Desarrollar un frontend funcional en JavaScript vanilla (sin frameworks)
 - ✓ Containerizar la aplicación completa con Docker Compose
 - ✓ Implementar validaciones, manejo de errores y casos de carga
 - ✓ Documentar el proyecto de forma profesional
-

Requisitos previos

-  **Node.js** v20+ y npm 10+
-  **Docker** 24+ y **Docker Compose** V2
-  **Ollama** instalado localmente con modelo descargado

- Instalar: `ollama pull mistral` (recomendado) o `ollama pull llama2`
- Verificar: `ollama serve` debe responder en `http://localhost:11434`
-  **Visual Studio Code** (o editor preferido)
-  **Git** instalado
-  **Extensión REST Client** en VS Code (para validacion.http)

 Estructura del proyecto sugerida pero usar la que consideréis mejor.

```
generador-preguntas-ia-[nombre-iniciales]/
|
├── backend/
│   ├── server.js           # Servidor Express (punto de entrada)
│   ├── routes.js          # Todas las rutas de la API
│   ├── services.js        # Lógica: Ollama + base de datos
│   ├── db.js              # Inicialización de SQLite3
│   ├── prompts.js         # Definición de los 3 temas y prompts
│   └── db/
│       └── preguntas.db    # Base de datos SQLite3 (generada)
│   ├── .env
│   ├── .env.example
│   └── package.json
|
├── frontend/
│   ├── index.html         # Estructura HTML
│   ├── style.css          # Estilos CSS (vanilla)
│   ├── main.js            # Lógica JavaScript
│   └── images/            # (Opcional) Capturas o recursos
|
├── validacion.http        # Tests de endpoints
├── docker-compose.yml     # Orquestación de servicios
├── README.md              # Documentación completa
├── checklist.md           # Control de progreso
├── .gitignore
└── package.json (raíz, opcional)
```

Parte 1: Configuración inicial del proyecto

1.1 🏠 Inicialización del proyecto

- ☐ Crear carpeta: `generador-preguntas-ia-[tu-nombre-iniciales]`
- ☐ Inicializar repositorio Git: `git init`
- ☐ Crear rama de trabajo: `git checkout -b hito2/desarrollo-ia`

1.2 📦 Backend - Instalación de dependencias

- ☐ Completar datos en `package.json` :
 - Nombre: `generador-preguntas-backend-[tu-nombre]`
 - Descripción: "Backend para generador de preguntas con Ollama"
 - Author: Tu nombre
- ☐ Instalar dependencias necesarias:

```
npm install express dotenv better-sqlite3 cors
npm install --save-dev nodemon
```

¿Qué hace cada paquete?

- `express` : Framework web
- `dotenv` : Cargar variables de entorno
- `better-sqlite3` : Base de datos SQL más simple que sqlite3
- `cors` : Permitir solicitudes desde el frontend
- `nodemon` : Reiniciar servidor al cambiar archivos (desarrollo)

1.3 ⚙️ Configuración de package.json (backend)

- ☐ Añadir `"type": "module"` para usar ES Modules
- ☐ Configurar scripts:

```
"scripts": {
  "dev": "nodemon server.js",
  "start": "node server.js"
}
```

1.4 🛠️ Archivos de configuración (backend)

- ☐ Crear `.env` `.env.example` `.gitignore` en raíz del proyecto: ``

1.5 📁 Frontend - Estructura básica

- ☐ En `frontend/`, crear `index.html`:
 - Estructura HTML5 semántica
 - Un `<div id="app">` como contenedor principal
 - Link a `style.css` y `main.js`
- ☐ Crear `style.css` vacío (lo rellenaremos después)
- ☐ Crear `main.js` vacío (lo rellenaremos después)

1.6 🗄 Base de datos SQLite3

- ☐ Crear carpeta `backend/db/`
- ☐ **NO** crear `preguntas.db` a mano
- ☐ El script `db.js` la generará automáticamente con las tablas

🖥 Parte 2: Backend - Implementación

2.1 📄 backend/db.js - Inicializar base de datos

- ☐ Crear archivo `backend/db.js` que:
 - Importe `better-sqlite3`

2.2 📝 backend/prompts.js - Definir los 3 o 4 temas

- ☐ Crear archivo `backend/prompts.js` con estructura: (ejemplo para 1 tema, no tomar al pie de la letra que os conozco ☹!!!!)

```
export const temas = [
  {
    id: 'javascript',
    nombre: 'JavaScript Avanzado',
    descripcion: 'Preguntas sobre ES6+, async/await, promesas',
    prompt: `Eres un profesor experto de JavaScript.
      Genera {num_preguntas} preguntas de opción múltiple
      sobre {subtema}. Incluye 4 opciones y marca la correcta.
      Formato JSON: { preguntas: [{ pregunta, opciones, correcta
    }, ...] }`
  },
  ...
],
```

```
.....  
];
```

- ☐ Cada tema debe tener:
 - `id` : Identificador único
 - `nombre` : Nombre legible
 - `descripcion` : Breve descripción
 - `prompt` : Template con placeholders {num_preguntas} y {subtema}

2.3 backend/services.js - Lógica principal

Este es el archivo más importante. Debe incluir:

Función: `generarPreguntas(tema, numPreguntas, subtema)`

- ☐ Recibir parámetros: tema, numPreguntas (2-5), subtema (opcional)
- ☐ Buscar tema en `prompts.js`
- ☐ Construir prompt completo reemplazando placeholders
- ☐ Realizar petición POST a Ollama en `http://localhost:11434/api/generate`
- ☐ Parsear respuesta JSON del modelo
- ☐ Guardar preguntas en BD (tabla preguntas)
- ☐ Devolver preguntas guardadas

Función: `obtenerPreguntas(tema)`

- ☐ Recibir tema
- ☐ Buscar en BD todas las preguntas del tema
- ☐ Devolver array de preguntas

Función: `eliminarPregunta(id)`

- ☐ Recibir ID de pregunta
- ☐ Eliminar de BD
- ☐ Devolver confirmación

Función: `limpiarTema(tema)`

- ☐ Eliminar todas las preguntas de un tema
- ☐ Devolver número de preguntas eliminadas

Consideraciones importantes:

-  **Manejo de errores:** Si Ollama no responde → mensaje claro

- 🕒 **Timeout:** Si Ollama tarda > 60 segundos → error
- 🔍 **Validaciones:** numPreguntas entre 1-5, tema válido, etc.

2.4 🌐 backend/routes.js - Endpoints de la API

- ☐ Crear archivo `backend/routes.js` que importe Express Router
- ☐ Implementar estos endpoints:

+ `POST /api/generate`

- **Recibe:** `{ tema, numPreguntas, subtema }`
- **Responde:** `{ success: true, preguntas: [...], mensaje: "..." }`
- **Error:** `{ success: false, error: "...", codigo: 400 }`

📋 `GET /api/preguntas`

- **Query params:** `?tema=javascript` (opcional)
- **Responde:** Array de preguntas (todas o filtradas)

🔍 `GET /api/preguntas/:id`

- **Parámetro:** ID de pregunta
- **Responde:** Pregunta específica o error 404

🗑️ `DELETE /api/preguntas/:id`

- **Parámetro:** ID de pregunta
- **Responde:** `{ success: true, mensaje: "Eliminada" }`

🧹 `DELETE /api/preguntas/tema/:tema`

- **Parámetro:** Nombre del tema
- **Responde:** `{ success: true, eliminadas: 5 }`

📊 `GET /api/temas`

- **Responde:** Lista de 3 temas disponibles

🏠 `GET /api/health`

- **Responde:** Estado del servidor y conexión a Ollama
- **Formato:** `{ status: "ok", ollama: "connected", timestamp: "..." }`

Todas las rutas deben:

- Validar entrada
- Usar try-catch

- Devolver JSON estructurado
- Incluir códigos HTTP adecuados (200, 400, 404, 500)

2.5 📄 backend/server.js - Servidor Express

- ☐ Crear archivo `backend/server.js` que:
 - Importe dotenv y cargue `.env`
 - Importe Express, db.js y routes.js
 - Cree aplicación Express
 - Configure CORS (permitir origen `http://localhost:5173` y similares)
 - Monte las rutas en `/api`
 - Sirva archivos estáticos del frontend en `/`
 - Escuche en puerto especificado en `.env`
 - Log en consola: "Servidor ejecutándose en puerto X"
-



Parte 3: Frontend - JavaScript Vanilla

3.1 📄 frontend/index.html

- ☐ Estructura HTML5 semántica con:
 - Título del proyecto
 - Selector de tema (dropdown con los 3 temas)
 - Input para número de preguntas (1-5)
 - Botón "Generar Preguntas"
 - Indicador de carga (spinner o texto "Generando...")
 - Contenedor para mostrar preguntas
 - Botón "Limpiar" para vaciar
- ☐ Debe cargar:
 - `<link rel="stylesheet" href="style.css">`
 - `<script src="main.js" type="module"></script>`

3.2 🎨 frontend/style.css

- ☐ Estilos básicos (CSS vanilla, sin frameworks):
 - Body: fondo, familia fuentes, márgenes

- Contenedor principal: centrado, ancho máximo 800px
- Selector tema: dropdown con estilos
- Input números: validación visual
- Botones: hover, active, disabled
- Contenedor preguntas: tarjetas con sombra
- Pregunta individual: claro, legible, espaciado
- Indicador de carga: spinner o animación
- Estado de error: texto rojo, icono
- Responsive: funcione en móvil (media queries)

No usar Tailwind ni Bootstrap. CSS puro.

3.3 ✂ frontend/main.js - Lógica JavaScript

- ☐ **Función:** `cargarTemas ()`
 - ☐ Hacer GET a `/api/temas`
 - ☐ Rellenar selector dinámicamente
 - ☐ Ejecutar al cargar página
- ☐ **Función:** `generarPreguntas ()`
 - ☐ Validar que tema está seleccionado
 - ☐ Validar que numPreguntas entre 1-5
 - ☐ Mostrar indicador de carga
 - ☐ Hacer POST a `/api/generate` con datos
 - ☐ Al recibir respuesta → mostrar preguntas
 - ☐ Si error → mostrar mensaje error claro
 - ☐ Ocultar indicador de carga
- ☐ **Función:** `mostrarPreguntas (preguntas)`
 - ☐ Limpiar contenedor
 - ☐ Para cada pregunta:
 - Crear elemento HTML con pregunta
 - Mostrar opciones si las hay
 - Añadir botón "Eliminar" para cada una
 - ☐ Estilizar como tarjetas
- ☐ **Función:** `eliminarPregunta (id)`

- ☐ Hacer DELETE a `/api/preguntas/{id}`
- ☐ Recargar lista de preguntas
- ☐ Mostrar confirmación
- ☐ **Función:** `limpiar()`
 - ☐ Limpiar contenedor
 - ☐ Resetear inputs
 - ☐ Hacer DELETE a `/api/preguntas/tema/{tema}` si es necesario
- ☐ **Event listeners:**
 - ☐ Click botón "Generar" → `generarPreguntas()`
 - ☐ Click botón "Limpiar" → `limpiar()`
 - ☐ Change tema → (opcional) cargar preguntas existentes
 - ☐ Input numPreguntas → validar rango 1-5

Consideraciones:

- ✓ Usar `fetch()` con async/await
- ✓ Try-catch para errores
- ✓ Mostrar/ocultar indicadores de carga
- ✓ Mensajes de error legibles
- ✓ No permitir envío si faltan datos



Parte 4: Testing y Validación

4.1 validacion.http - Tests de endpoints

- ☐ Crear archivo `validacion.http` en raíz del proyecto con tests para todos los endpoints
- ☐ Incluir secciones para cada endpoint:

4.2 ✓ Checklist de pruebas manuales

- ☐ ✓ Backend levanta sin errores
- ☐ ✓ Ollama responde en `http://localhost:11434`
- ☐ ✓ Frontend carga en `http://localhost:3000`
- ☐ ✓ Selector de temas funciona
- ☐ ✓ Generar preguntas JavaScript funciona

- ☐ ✓ Generar preguntas Seguridad funciona
- ☐ ✓ Generar preguntas Normativa funciona
- ☐ ✓ Las preguntas se guardan en BD
- ☐ ✓ Las preguntas se muestran en frontend
- ☐ ✓ Eliminar pregunta funciona
- ☐ ✓ Limpiar tema funciona
- ☐ ✓ Validaciones: bloquea numPreguntas fuera de rango
- ☐ ✓ Validaciones: bloquea envío sin tema
- ☐ ✓ Mensaje de error si Ollama no responde
- ☐ ✓ Indicador de carga se muestra/oculta correctamente

Parte 5: Dockerización

5.1 docker-compose.yml

- ☐ Crear archivo en raíz del proyecto que orqueste:

5.2 Levantar con Docker Compose

- ☐ Comando para construir y levantar:

```
docker compose up --build
```

- ☐ Verificar servicios:

- Backend: `http://localhost:3000` o el puerto definido en `.env`
- Ollama: `http://localhost:11434`

- ☐ Comando para parar:

```
docker compose down
```

Parte 6: Documentación (README.md)

- ☐ Crear `README.md` completo que incluya:

6.1 Descripción general

- Qué es el proyecto
- Casos de uso
- Tecnologías usadas

6.2 Requisitos

- Node.js, Docker, Ollama, etc.

6.3 Instalación (desarrollo local)

```
# Backend
cd backend
npm install
cp .env.example .env
npm run dev

# Frontend (otra terminal)
# Simplemente servir estáticos o usar python -m http.server 5173
```

6.4 Instalación (con Docker)

```
docker compose up --build
# Acceder a http://localhost:3000
```

6.5 Estructura de carpetas

- Explicar cada fichero y su responsabilidad

6.6 API Endpoints

- Para cada endpoint: método, ruta, parámetros, respuesta ejemplo

6.7 Extensiones futuras

- Estadísticas de preguntas
- Exportar a PDF



Parte 7: Git y Control de versiones

7.1 Commits incrementales

- ☐ Commit inicial: "init: estructura básica del proyecto"
- ☐ Commit backend: "feat: backend Express con rutas"
- ☐ Commit servicios: "feat: integración Ollama y SQLite3"
- ☐ Commit frontend: "feat: interfaz JavaScript vanilla"
- ☐ Commit docker: "feat: dockerización con compose"
- ☐ Commit docs: "docs: README completo"

Mensajes de commit deben ser descriptivos:

```
feat: agregar endpoint /api/generate  
fix: corregir error de parseo JSON en Ollama  
docs: actualizar README con ejemplos
```

7.2 Rama de trabajo

- ☐ Crear rama: `git checkout -b hito2/desarrollo-ia`
- ☐ Hacer commits en rama
- ☐ Al terminar, crear Pull Request hacia `main` para que valide el profesor/a

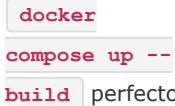
7.3 Pull Request

- ☐ Título: "Entrega: Generador de Preguntas con Ollama"
- ☐ Descripción incluya:
 - Resumen de lo implementado
 - División del trabajo (si es en pareja)
 - Dificultades y cómo se resolvieron
 - Instrucciones para probar

III Rúbrica de Evaluación



Criterios y Pesos

#	Criterio	Peso	● Nivel 1	● Nivel 2	● Nivel 3	Nota
1	 Configuración Backend	1.5	Dependencias incompletas, .env mal configurado	Configuración completa pero con errores menores	Perfecta, Express levanta sin errores	☐ ___/1.5
2	 Integración Ollama	2.0	No se conecta a Ollama, sin manejo errores	Se conecta pero falla con entradas especiales	Conexión robusta, buen manejo errores	☐ ___/2.0
3	 Base de Datos SQLite3	1.5	No persiste datos, BD no se crea	Persiste pero sin validaciones	Persistencia completa, validaciones OK	☐ ___/1.5
4	 Endpoints API	2.0	< 4 endpoints, no validan entrada	6+ endpoints pero con errores	Todos endpoints funcionales, JSON correcto	☐ ___/2.0
5	 Frontend JavaScript Vanilla	2.0	No carga, sin funcionalidad	Carga pero interfaz incompleta/confusa	Interfaz clara, funcional, responsive	☐ ___/2.0
6	 Validaciones y Errores	1.5	Sin validaciones	Validaciones básicas pero incompletas	Validaciones completas, mensajes claros	☐ ___/1.5
7	 Dockerización	1.5	docker-compose no funciona	Levanta con errores menores	 perfecto	☐ ___/1.5
8	 Documentación README	1.0	Falta o muy incompleta	Completa pero explicaciones vagas	Detallada, ejemplos, instrucciones claras	☐ ___/1.0
9	 Tests (validacion.http)	0.8	No existe o sin endpoints	Algunos endpoints, sin comentarios	Todos endpoints, bien organizados	☐ ___/0.8
10	 Git y Workflow	0.7	< 3 commits, mensajes genéricos	Commits regulares, mensajes OK	Commits descriptivos, workflow limpio	☐ ___/0.7

TOTAL MÁXIMO: 15.0 puntos



Cálculo de la nota final

 **Nota Final = Σ (Puntos obtenidos por criterio)**

 **Escala de niveles:**

- **Sin realizar** = 0 puntos
- **Nivel 1** = 33% del peso máximo del criterio
- **Nivel 2** = 66% del peso máximo del criterio
- **Nivel 3** = 100% del peso máximo del criterio

-

Entrega

 **Método:** Pull Request en GitHub

 **Plazo:** [Definir por el profesor]

Requisitos obligatorios:

- ✓ PR creado antes de la fecha límite
 - ✓ docker-compose.yml funciona sin errores
 - ✓ Archivo `validacion.http` con tests
 - ✓ README.md completo y claro
 - ✓ Todos los `.env` NO están versionados (solo `.env.example`)
 - ✓ Checklist.md completado
 - ✓ **Exposición en clase** del proyecto funcionando.
-

Trabajo en pareja (obligatorio)

- ☐ Ambos deben aparecer como autores en commits (Co-authored-by)
 - ☐ Debe haber clara división de responsabilidades:
 - **Persona A:** Backend (server.js, routes.js, services.js)
 - **Persona B:** Frontend (index.html, style.css, main.js)
 - Ambos: dockerización y documentación
 - ☐ Ambos deben entender todo el código
 - ☐ PR debe indicar quién hizo qué
-